ASE CHAPTER 1

T1 · Multiple Choice Questions

Official MCQs 1.1–1.20 with sourced answers · plus an authored set in the same pattern

Rapid self-test — recall prompts (no answers shown)

> Cover the cards above, then answer these from memory. If any item fails, return to its source page.

Define software in its three-component form.
Name the two families of manuals and give two members of each.
List the four software characteristics.
Distinguish product from process in one sentence each.
List the three process metrics and three product metrics.
State the unit of effort, and justify why it is neither "persons" nor "months".
Name the four factors of management dependency **in the required order**.
State the two definitions of software engineering quoted in the text.
Give the four reasons software process improvement fails.
State the percentage of development projects attributed to customized products.

Retrieval set — Part A · Official MCQs (1.1 – 1.20)

1.1 · Software is

(a) Superset of programs · **(b)** Subset of programs · **(c)** Set of programs · **(d)** None of the above

Superset of programs (a) — Answer

Software = Program + Documentation + Operating Procedures. Programs are one component inside software — software is the **superset**, the program the **subset**. Option (b) reads the same relationship backwards.

Aggarwal, §1.2.2, pp.4–5 (Fig. 1.1).

1.2 · Which is NOT the part of operating procedure manuals?

(a) User manuals · **(b)** Operational manuals · **(c)** Documentation manuals · **(d)** Installation manuals

Documentation manuals (c) — Answer

Two separate families.

Operating procedure manuals: user manuals, operational manuals, system overview, beginner's guide tutorial, reference guide, installation guide, system administration guide.

Documentation manuals: analysis/specification, design, implementation, testing, formal specification, context diagram, data flow diagrams, flow charts, entity-relationship diagram, source code listings, cross-reference listing, test data, test results.

Aggarwal, §1.2.2, pp.5–6 (Fig. 1.2, Fig. 1.3).

1.3 · Which is NOT a software characteristic?

(a) Software does not wear out · **(b)** Software is flexible · **(c)** Software is not manufactured · **(d)** Software is always correct

Software is always correct (d) — Answer 📝

Characteristics: software **does not wear out**, software **is not manufactured**, software **is flexible**, and its components are **reusable**. "Always correct" is not a property — it is the exact opposite of what the myths section later warns about.

Aggarwal, §1.2.4, pp.8–10.

ليش (d) هي الفخ ⚡

باقي الخيارات كلها خصائص حقيقية منصوصة، فالسؤال يكافئ اللي **حفظ القائمة**. و«always correct» تسمع مثل مدح — لهذا تعدي على قراءة سريعة. حالات الفشل المذكورة مثل Patriot وY2K توضح عملياً أن البرمجيات مو دائماً صحيحة.

Aggarwal, §1.2.4, pp.8–10; §1.1.1, pp.2–3.

1.4 · Product is

(a) Deliverables · **(b)** User expectations · **(c)** Organization's effort in development · **(d)** None of the above

Deliverables (a) — Answer 📝

"Product: what is delivered to the customer... Basically, it is nothing but a set of deliverables only." Option (c) — the organization's effort in development — is the definition of **process**, not product.

Aggarwal, §1.5.2, p.13.

1.5 · To produce a good quality product, process should be

(a) Complex · **(b)** Efficient · **(c)** Rigorous · **(d)** None of the above

Efficient (b) — Answer 📝

"An efficient process is required to produce good quality products." "If the process is weak, the end product will undoubtedly suffer, but an obsessive over-reliance on process is also dangerous."

Aggarwal, §1.5.2, p.13.

1.6 · Which is NOT a product metric?

(a) Size · **(b)** Reliability · **(c)** Productivity · **(d)** Functionality

Productivity (c) — Answer 📝

"Examples of process metrics include productivity, quality, failure rate, efficiency etc. Examples of product metrics are size, reliability, complexity, functionality etc."

Productivity is a process metric, so it is the one that does not belong. Note that **functionality** is a product metric — which is exactly why it cannot also be the answer here.

Aggarwal, §1.5.4, p.13.

1.7 · Which is NOT a process metric?

(a) Productivity · **(b)** Functionality · **(c)** Quality · **(d)** Efficiency

Functionality (b) — Answer

"Examples of process metrics include productivity, quality, failure rate, efficiency etc." Question 1.6 read in reverse.

Aggarwal, §1.5.4, p.13.

قاعدة واحدة تحل 1.6 و1.7 و1.19 من التمارين

اسأل: **مَن المُقاس؟** إذا القياس يوصف **نشاط البناء** ⇒ مقياس عملية. وإذا يوصف **الشي المبني** ⇒ مقياس منتج. والـproductivity دائماً عملية لأنه **معدّل** (ناتج ÷ جهد).

Aggarwal, §1.5.4–1.5.5, pp.13–14.

1.8 · Effort is measured in terms of:

(a) Person-months · **(b)** Rupees · **(c)** Persons · **(d)** Months

Person-months (a) — Answer

"Most appropriate unit of effort is Person Months (PMs), meaning thereby number of persons involved for specified months." Option (c) counts people, option (d) counts time — effort is the **product** of the two, which is why neither alone works.

Aggarwal, §1.5.5, p.14.

1.9 · UML stands for

(a) Uniform modeling language · **(b)** Unified modeling language · **(c)** Unit modeling language · **(d)** Universal modeling language

Unified modeling language (b) — Answer

"Another definition from unified modeling language (UML) [OMG2K] is: 'A component represents a modular, deployable, and replaceable part of a system that encapsulates implementation and exposes a set of interfaces.'"

Aggarwal, §1.5.6, p.14.

1.10 · An independently deliverable piece of functionality providing access to its services through interface is called

(a) Software measurement · **(b)** Software composition · **(c)** Software measure · **(d)** Software component

Software component (d) — Answer

"An independently deliverable piece of functionality providing access to its services through interfaces." "A component represents a modular, deployable, and replaceable part of a system that encapsulates implementation and exposes a set of interfaces."

Aggarwal, §1.5.6, p.14.

⚠ **هذا السؤال مطبوع `1.1` بسلايد 58 — رقم مكرر**

فهرسته هنا **1.10** حتى يمشي التسلسل 1.1 ← 1.20 بلا فراغ.

Aggarwal, §1.5.6, p.14.

1.11 · Infrastructure software are covered under

(a) Generic products · **(b)** Customized products · **(c)** Generic and Customized products · **(d)** None of the above

Generic products (a) — Answer

"Generic products are developed for anonymous customers... Infrastructure software like operating system, compilers, analyzers, word processors, CASE tools etc. are covered in this category."

Aggarwal, §1.5.7, p.14.

1.12 · Management of software development is dependent on

(a) people · **(b)** product · **(c)** process · **(d)** all of the above

all of the above (d) — Answer

"The management of software development is heavily dependent on four factors: People, Product, Process, and Project." Options (a), (b) and (c) are each one of them.

Aggarwal, §1.6, p.15.

1.13 · During software development, which factor is most crucial?

(a) People · **(b)** Product · **(c)** Process · **(d)** Project

People (a) — Answer

"The management of software development is heavily dependent on four factors: People, Product, Process, and Project." Order of dependency (Fig. 1.8): **People (1)** → **Product (2)** → **Process (3)** → **Project (4)**. People comes first, so it is the most crucial factor.

"The process priority is after people and product, however, it plays very critical role for the success of the project."

Aggarwal, §1.6, p.15; §1.6.3, p.16; Fig. 1.8, p.15.

1.14 · Program is

(a) subset of software · **(b)** super set of software · **(c)** software · **(d)** none of the above

subset of software (a) — Answer

Software = Program + Documentation + Operating Procedures.

Aggarwal, §1.2.2, pp.4–5 (Fig. 1.1).

1.15 · Milestones are used to

(a) know the cost of the project · **(b)** know the status of the project · **(c)** know user expectations · **(d)** none of the above

know the status of the project (b) — Answer

"The milestones are the events that are used to ascertain the status of the project... The milestones are essential for project planning and management." Cost belongs to estimation, and user expectations belong to requirements — neither is what a milestone reports.

Aggarwal, §1.5.1, p.13.

1.16 · The term module used during design phase refers to

(a) Function · **(b)** Procedure · **(c)** Sub program · **(d)** All of the above

All of the above (d) — Answer

No single definition is fixed; a module may be **"a FORTRAN subroutine"**, **"an Ada package"**, **"procedures and functions of PASCAL and C"**, **"C++/Java classes"**, **"Java packages"**, or even **"a work assignment for an individual developer"** — **"all these definitions are correct."**

Aggarwal, §1.5.6, p.14.

1.17 · Software consists of

(a) Set of instructions + operating system · **(b)** Programs + documentation + operating procedures · **(c)** Programs + hardware manuals · **(d)** Set of programs

operating procedures + documentation + Programs (b) — Answer

Software = Program + Documentation + Operating Procedures (Fig. 1.1, p.4). Options (a), (c) and (d) each drop two of the three components.

Aggarwal, §1.2.2, pp.4–5 (Fig. 1.1).

1.18 · Software engineering approach is used to achieve:

(a) Better performance of hardware · **(b)** Error free software · **(c)** Reusable software · **(d)** Quality software product

Quality software product (d) — Answer

Schach: **"a discipline whose aim is the production of** quality software**, software that is delivered on time, within budget, and that satisfies its requirements." "The art of programming only is no longer sufficient to construct large programs."**

Aggarwal, §1.2 and §1.2.1, p.4.

1.19 · Concept of software engineering are applicable to

(a) Fortran language only · **(b)** Pascal language only · **(c)** 'C' language only · **(d)** All of the above

All of the above (d) — Answer

The two definitions rest on **engineering principles** and on the **process** of producing quality software — they name no programming language, so the concepts are not tied to any one language.

Aggarwal, §1.2.1, p.4; Answers, p.498.

سبب اختيار (d)

التعريفان ما يذكران لغةً معينة؛ والمفتاح الرسمي يؤكد الخيار (d).

1.20 · CASE Tool is

(a) Computer Aided Software Engineering · **(b)** Component Aided Software Engineering · **(c)** Constructive Aided Software Engineering · **(d)** Computer Analysis Software Engineering

Computer Aided Software Engineering (a) — Answer

"Many people (especially CASE tool vendors) believe that CASE (Computer Aided Software Engineering) tools represent the so-called silver bullet..." "Operating systems, compilers, analysers, word processors, CASE tools etc."

Aggarwal, §1.1.2, p.3; §1.5.7, p.14.

Part B · Authored MCQs (same pattern)

B.1 · The definition of "software" differs from the everyday use of the word mainly because it includes:

(a) the hardware it runs on · **(b)** the documentation and operating procedures · **(c)** the development team · **(d)** the licence agreement

the documentation and operating procedures (b) — Answer

Software = Program + Documentation + Operating Procedures. The everyday sense of "software" is programs; the definition is a three-part construct, which is why a program alone is not a software product.

Aggarwal, §1.2.2, pp.4–5 (Fig. 1.1).

B.2 · Which of the following belongs to the operating procedure manuals rather than to the documentation manuals?

(a) Design specification · **(b)** Data-flow diagram · **(c)** Installation guide · **(d)** Test results

Installation guide (c) — Answer

The two families are kept strictly apart. Design specifications, data-flow diagrams and test results are all **documentation**; the installation guide is listed among the **operating procedure** manuals.

Aggarwal, §1.2.2, pp.5–6 (Fig. 1.2, Fig. 1.3).

B.3 · A software house builds a payroll system for one specific ministry. Under the classification this product is:

(a) generic · **(b)** bespoke (customized) · **(c)** infrastructure · **(d)** open source

(customized) bespoke (b) — Answer

"The customized products are developed for particular customers." About 80% of development projects fall in this category.

Aggarwal, §1.5.7, p.14.

B.4 · Software process improvement commonly fails for all of the following reasons EXCEPT:

(a) not enough time · **(b)** lack of knowledge · **(c)** wrong motivations · **(d)** the absence of a CASE tool

the absence of a CASE tool (d) — Answer

Four reasons software process improvement is difficult: **"not enough time"**, **"lack of knowledge"**, **"wrong motivations"** and **"insufficient commitment"**. Tooling is not among them, so (d) is the exception.

Keep this separate from the different question of why CASE tools cannot end the crisis — that argument belongs to §1.1.2 and rests on the no-silver-bullet claim, not on this list. The two sections answer two different questions.

Aggarwal, §1.2.3, pp.6–7.

B.5 · Which statement is one of the developer-perspective software myths?

(a) The tested code is the only deliverable of a project · **(b)** A prototype reduces requirements risk · **(c)** Maintenance costs dominate the life-cycle cost · **(d)** Requirements change during development

The tested code is the only deliverable of a project (a) — Answer

Developer-perspective myth: **"Tested code is only one of the deliverable!"** Options (b), (c) and (d) are supported positions, not myths.

Aggarwal companion slides, slide 45 (Software Myths — Developer Perspectives).

B.6 · The aim of software development has shifted from producing working programs to producing:

(a) smaller programs · **(b)** good quality maintainable programs · **(c)** programs with more features · **(d)** error-free programs

good quality maintainable programs (b) — Answer

"The aim has been shifted from developing working programs to good quality, maintainable programs." The change was driven by the **"cost of maintaining software"**.

Aggarwal, §1.4, p.12; §1.2.1, p.4.

B.7 · "The strength and stability of a bridge can be calculated with reasonable precision" is listed as a difference between building a bridge and writing a program. The corresponding statement for a program is:

(a) the correctness of a program can be calculated with existing methods · **(b)** it is not possible to calculate the correctness of a program with existing methods · **(c)** every program is a repeat of a previous program · **(d)** the requirements for a program do not change during construction

program with existing it is not possible to calculate the correctness of a (b) — Answer methods

The bridge-versus-program comparison pairs every engineering strength with a software weakness: the bridge problem is **well understood**, the program problem is **only partly understood**; bridge requirements **barely change**, program requirements **change in all phases**; bridges are **calculable**, programs are **not**.

Aggarwal, §1.2.4, Table 1.1, p.8.

B.8 · If productivity is defined as output per unit of effort, and effort is measured in person-months, then productivity is measured in:

(a) lines of code · **(b)** person-months · **(c)** lines of code per person-month · **(d)** person-months per line of code

lines of code per person-month (c) — Answer

"Productivity may be measured as LOC/PM (lines of code produced/person month)." Option (d) inverts the ratio — a classic trap, because it is the **effort per unit of output** (the inverse rate), not productivity.

Aggarwal, §1.5.5, p.14.
